Runtime Configuration and Cache Inva

Abstract. This paper is about the gap between changing a configuration value and actually changing system behaviour. The gap is almost always a cache, and the failure mode is silent: the setting appears saved while the running process keeps using the previous value.

1 Introduction. Configuration is usually introduced for one reason: to avoid rebuilding for behaviour that changes. If reading a value requires a restart, the configuration has failed at its only job, and operators will work around it in ways that hide the real state.

2 Precedence. In practice a value can come from several places at once: a session level override, a persisted settings file, and an environment variable. The order matters more than the individual values, because environment variables are weaker than persisted runtime settings.

3 Single source of truth. the effective value must be read from a single source of truth. Duplicating a default in three modules guarantees that one of them will drift, and the drift will be discovered only when the three disagree in production, which is the worst possible time.

4 Cache invalidation. a configuration change that does not invalidate the cache is silently ignored. The classic symptom is an operator changing a value, observing no effect, and concluding that the feature is broken. Clearing the cached component must be part of the same write path that stores the new value.

5 What to invalidate. Configuration rarely affects one object only. A retrieval pipeline may hold a schema, an index, and a learned component, each built at a different time. Invalidating the outermost object while leaving the inner ones alive produces a state that is neither the old configuration nor the new one.

6 Rollback. a rollback switch should be reachable without a rebuild. Quality problems are discovered during use, and the window for a safe rollback is measured in minutes. A rollback that requires a build and a deployment is equivalent to no rollback at all.

Table 1 Scope of configuration keys. session level, highest priority; settings file, persisted; environment variable, lowest; hard coded default, fallback only.

7 Validation. A value that is accepted but unusable is worse than a value that is rejected. Ranges, enumerations and mutually exclusive combinations should be checked on write, and the error should name the offending key rather than reporting a generic failure.

8 Observability. the report must record which configuration produced it. Without that record, two reports can differ by a factor of two and the discussion degenerates into speculation about which run was which. Recording the configuration turns an argument into a lookup.

9 Secrets. secrets belong in the environment while behaviour belongs in settings. Behaviour changes should be reviewable and versionable, while credentials should never be written into a file that is meant to be shared. Mixing the two forces every operator to handle secrets merely to change a threshold.

10 Migration of stored values. A settings file written by an older version may contain keys that no longer exist, or omit keys that are now required. Loading must tolerate unknown keys and fill missing ones from defaults, otherwise one upgrade renders the whole file unreadable.

11 Testing configuration. Configuration logic deserves tests as much as any other logic. The two cheapest and most valuable ones assert the precedence order and assert that every declared key has a default, so a forgotten key fails immediately instead of at runtime.

12 Failure modes. The three recurring failures are: a value that never takes effect, a v

alue that takes effect only after restart, and a value that takes effect but is reported differently from what is used. Only the last one is invisible to the operator, and it is the most damaging because it corrupts the evidence.

13 Conclusion. Treat configuration as part of the system rather than as an operational afterthought: define the precedence, keep one source of truth, invalidate aggressively, and record the effective value wherever results are reported.